Adapting LLM Agents to Interactive Reasoning through Parallel Exploration

Yunfei Xie[1], Shiyi Lan[2], Chen Wei[1]

[1]**Rice University** , [2]**NVIDIA**

Abstract

ARC-AGI-3 is an interactive reasoning benchmark requiring agents to discover novel game mechanics and complete multi-level tasks through trial-and-error exploration. Frontier models fail dramatically, exhausting large action budgets with almost no progress, while humans solve the same games with ease. We attribute this gap to fundamentally novel mechanics that lie outside pretrained knowledge and extremely sparse learning signals where only level completion provides feedback. To address these challenges, we propose two complementary techniques. Multi-turn Reinforcement Learning with Verifiable Rewards (RLVR) adapts the model online to each game by fine-tuning on level-completing trajectories collected during interaction. Level-synchronized parallel sampling runs k trajectories simultaneously and shares progress whenever any trajectory completes a level, substantially increasing the chance of obtaining a useful learning signal. The two techniques are complementary: levels that the base model cannot reach become reachable, and the required actions drop by up to 4×. Models trained on individual ARC-AGI-3 games also transfer to ALFWorld, nearly doubling baseline performance, suggesting that the learned reasoning capabilities are not merely game-specific.

1. Introduction

The Abstraction and Reasoning Corpus for Artificial General Intelligence (ARC-AGI), introduced in 2019, has become a fundamental benchmark for evaluating general fluid intelligence in artificial systems [Chollet, 2019]. The goal of ARC-AGI is to measure whether AI systems can truly generalize, testing their ability to adapt to novel tasks rather than demonstrating proficiency in narrow, pre-trained domains.

ARC-AGI has evolved through three versions. In ARC-AGI-1 and ARC-AGI-2 [Chollet, 2019, Chollet et al., 2026], players are shown several example grid transformations and must deduce the underlying rule to transform a new input grid correctly. ARC-AGI-3 shifts from static grid tasks to interactive benchmarks, with a central focus on *skill acquisition efficiency*: how quickly a system can learn new things when faced with novel situations requiring minimal prior knowledge [ARC Prize Foundation, 2026]. While AI systems can already achieve proficiency in narrow domains, the ability to learn something genuinely new remains out of reach. ARC-AGI-3 tests this by placing agents in game-like environments where they must discover goals and complete tasks through trial-and-error exploration over multiple steps. An agent that learns the game's rules faster will waste fewer actions on wrong attempts, so the number of actions spent to complete a level directly reflects how quickly it is acquiring knowledge of the game. Action efficiency therefore serves as a direct proxy for learning efficiency [Kamradt, 2025a].

Success on ARC-AGI-3 would demonstrate what has been termed "micro-AGI," exhibiting the right properties of general intelligence at a small scale that could theoretically be scaled up to real-world applications [Chollet and Knoop, 2025].

ARC-AGI-3 presents an extremely challenging benchmark for current AI systems. When placed in novel environments without instructions or hints, frontier models fail to explore effectively: they exhaust large action budgets without completing even basic levels, whereas humans solve the same games efficiently [Kamradt, 2025a,b]. In early evaluations, GPT-5 High Reasoning [OpenAI, 2025] on ls20, one of the games in ARC-AGI-3, exhibited repetitive, non-exploratory behavior. It executed numerous actions without completing even the first level [Kamradt, 2025a]. A subsequent 30-day agent competition evaluated frontier models including O3 High and Grok 4, which similarly failed to make meaningful progress despite consuming far more actions than humans [Kamradt, 2025b,a]. This large performance gap between humans and AI reveals core limitations in current AI approaches to interactive reasoning.

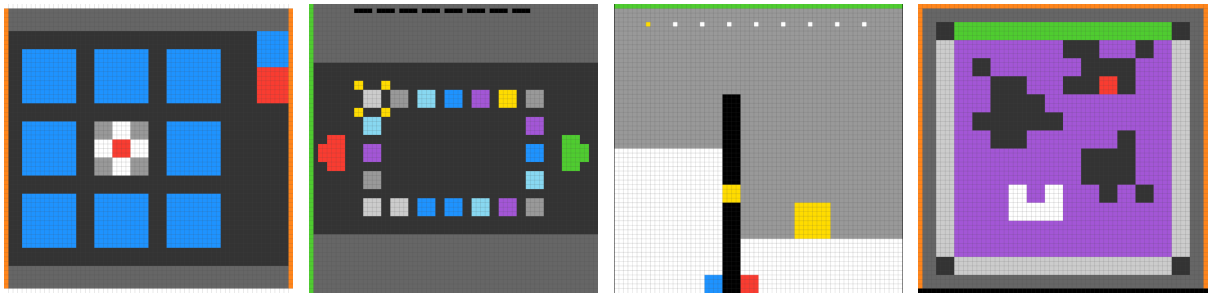
What causes this gap? We identify two primary sources of difficulty. First, the tasks are fundamentally novel. Each ARC-AGI-3 game is built on distinct mechanics designed to be fundamentally different from one another, and no instructions or goals are provided to the agent [Kamradt, 2025b]. The games rely only on core knowledge priors rather than specialized domain expertise, requiring the agent to adapt to unfamiliar environments through exploration alone [Chollet and Knoop, 2025]. Second, learning signals are extremely sparse. The only feedback ARC-AGI-3 provides is level completion itself, and most action sequences do not achieve it. Most individual actions either produce no useful feedback or trigger immediate failure such as a level reset. Since each level typically requires tens of sequential correct actions, successful trajectories are exceedingly rare. Extreme novelty and sparse rewards thus combine to make ARC-AGI-3 exceptionally hard, exposing core difficulties in interactive reasoning over long-horizon tasks.

To address these challenges, we propose two complementary techniques. We introduce multi-turn Reinforcement Learning with Verifiable Rewards (RLVR) to handle novel game mechanics: the model adapts online by fine-tuning on level-completing trajectories, building game-specific strategies through experience rather than prior knowledge. For sparse rewards, we extend pass@k from single-step generation to multi-step interactive tasks. A single trajectory, pass@1, has very low probability of completing a level, but running k parallel trajectories, pass@k, substantially increases the chance that at least one succeeds. We design a level-synchronized parallel sampling mechanism: whenever any trajectory completes a level, all others synchronize to that state, checkpointing progress rather than restarting from scratch.

Both components contribute meaningfully. On vc33, parallel sampling with level synchronization enables completion of all five levels, whereas the base model alone fails beyond Level 2. Adding RLVR further reduces the actions required to reach Level 5 by approximately 4×, from 79 to 19 steps. On ft09, RLVR enables Level 3 completion where parallel sampling alone fails. Furthermore, policies learned on individual ARC-AGI-3 games generalize to ALFWorld [Shridhar et al.], a text-based household task benchmark with entirely different mechanics. The overall success rate improves from 14.8% to up to 28.5%, nearly doubling baseline performance, suggesting that the learned reasoning skills extend well beyond the training environment.

In summary, our contributions are:

- We propose a multi-turn RLVR framework that fine-tunes the model on level-completing trajectories collected during online interaction, enabling adaptation to novel game mechanics without any prior knowledge of the game.
- We extend pass@k to multi-step interactive tasks and design a level-synchronized parallel sampling to counter the sparsity of learning signals, substantially increasing level



(a) ft09: Grid-based puzzle with spatial reasoning.

(b) lp85: Board-like environment with colored pieces.

(c) vc33: Orchestration game with liquid level coordination.

(d) as66: Shape manipulation on a purple canvas.

Figure 1 | **Example game environments from ARC-AGI-3.** Each game presents unique interactive reasoning challenges requiring agents to learn game mechanics through exploration without instructions. Agents must discover rules, plan actions, and complete multiple levels of increasing difficulty.

completion rates.

- We demonstrate significant improvements on ARC-AGI-3 tasks, reducing actions required by up to 4×, and show that learned capabilities transfer to other interactive reasoning benchmarks, nearly doubling success rates on ALFWorld.

2. ARC-AGI-3

2.1. Benchmark Overview

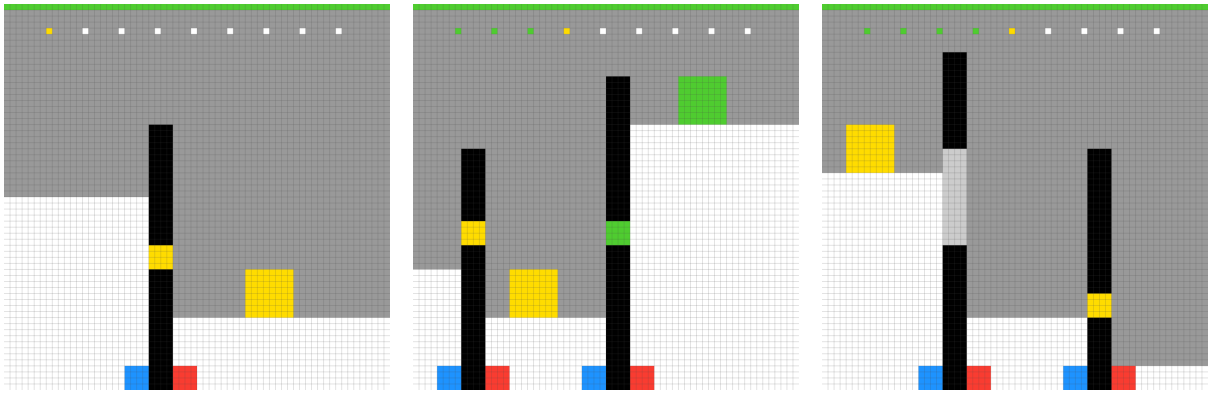
ARC-AGI-3 [ARC Prize Foundation, 2026] extends the original ARC challenge by shifting from static grid-based reasoning to interactive game environments that test an agent's ability to learn through exploration. Unlike traditional benchmarks that provide task descriptions or instructions, ARC-AGI-3 offers no textual hints or guidance, and agents must discover game mechanics, objectives, and winning conditions entirely through interaction.

The benchmark currently releases 6 publicly available game environments (`ft09`, `ls20`, `vc33`, `sp80`, `lp85`, `as66`), each containing between 8 and 10 levels with each subsequent level introducing new mechanics, as illustrated in Figure 1.

The benchmark's evaluation criterion prioritizes both effectiveness and efficiency: agents are scored primarily on the number of levels completed, with fewer total steps used counting as a secondary criterion among equally performing agents. This dual objective encourages solutions that not only discover winning strategies but do so with minimal exploration. Agents interact with the environment through a simple API: at each step, the agent receives an observation and submits an action, then receives the next observation along with a signal indicating whether the level was completed.

2.2. Observation and Action Spaces

Visual Observations. Agents receive visual observations as 64×64 pixel RGB frames with a discrete palette of 16 colors. Each frame contains both the game environment and a status bar displaying the number of steps remaining before an automatic level restart. When the step counter reaches zero, the current level resets to its initial state. In the majority of games, the number of levels passed is also displayed.



(a) Level 1: Single block, single target.

(b) Level 4: Two blocks, two targets.

(c) Level 5: Conditional target activation.

Figure 2 | **Progressive level design in vc33.** vc33 is an orchestration-type game where players coordinate liquid levels to position floating blocks at target heights. **Level 1** (left): A single yellow block must reach the yellow target marker on the column by adjusting the liquid level using blue/red controls. **Level 4** (middle): Two independent chambers require coordinating two liquid levels to align a yellow block and a green block with their respective colored targets. **Level 5** (right): A gray target zone is introduced that only becomes active when liquid levels on both sides fall within the gray region, adding conditional logic to the coordination task. Each level retains mechanics from earlier levels while introducing new elements, requiring cumulative understanding.

Action Spaces. The benchmark features three control schemes. Games such as `ls20` use arrow-based control with directional keyboard inputs (up, down, left, right), yielding an action space of size $|\mathcal{A}| = 4$. Games such as `ft09`, `vc33`, and `lp85` employ click-based control, allowing the agent to click any pixel location in the frame. This yields an action space of size $|\mathcal{A}| = 64 \times 64 = 4{,}096$. Games such as `sp80` and `as66` introduce combined control schemes that integrate both arrow and click inputs, resulting in action spaces of size $|\mathcal{A}| = 4{,}100$.

These large action spaces, combined with the long trajectories required for level completion, make successful trajectories combinatorially unlikely: each step involves selecting from thousands of possible actions, and each level demands tens of correct actions in sequence. This reinforces the sparse learning signal structure described in Section 1.

2.3. Task Structure and Mechanics

Levels within each game must be completed sequentially. Each attempt ends in one of two ways: the environment advances to the next level if the agent satisfies the winning conditions, or resets the current level to its initial state if the step limit expires.

Progressive Level Design. Within each game, levels progressively add new elements while retaining earlier ones. We illustrate this using vc33, shown in Figure 2, an orchestration-type game where players coordinate liquid levels to position floating blocks at target heights. In Level 1, the player adjusts a single liquid chamber using blue and red controls to raise one yellow block to its target height marked on the column. In Level 4, two independent chambers must be coordinated: a yellow block and a green block each need to reach their respective colored target markers, requiring the player to manage two liquid levels simultaneously. Level 5 introduces a new conditional mechanic: a gray horizontal band appears spanning both chambers, representing a "validation zone." The level is only considered complete when both

liquid levels simultaneously fall within this gray band, and the floating blocks must align with their target heights at that moment. This requires the player to reason about the joint state of multiple elements rather than adjusting each chamber independently.

This design reflects a deliberate curriculum structure within each game. Early levels serve as tutorials that teach core mechanics by introducing only one concept at a time. Difficulty then increases rapidly as the player must adapt to new situations with new rules [Kamradt, 2025a]. Each subsequent level retains previously learned mechanics while layering additional constraints. This structure tests whether agents can integrate cumulative knowledge, a key characteristic of efficient learning.

3. Related Work

Reinforcement Learning for LLM Agents. Reinforcement learning has become a central paradigm for training LLM agents in interactive environments. Early work applied classical algorithms such as DQN [Van Hasselt et al., 2016] to text-based games [Narasimhan et al., 2015], while later approaches adopted policy gradient methods like PPO [Schulman et al., 2017] for embodied tasks [Shridhar et al.] and web-based environments . More recently, several frameworks have addressed multi-turn RL for LLM agents. RAGEN [Wang et al., 2025] proposes StarPO, a trajectory-level GRPO variant that concatenates states, reasoning, and actions into episode-level responses, but faces scalability challenges in long-horizon settings. Search-R1 [Jin et al., 2025] trains LLMs to leverage search engines during multi-step reasoning via outcome-based rewards and retrieved token masking. LOOP [Chen et al., 2025] combines RLOO with PPO-style updates for long-horizon interactive agents. AgentGym-RL [Xi et al., 2025] introduces ScalingInter-RL for balancing exploration and exploitation across diverse tasks. Our work differs from these frameworks in two key respects: we operate in visual environments with fundamentally novel mechanics (no text-based instructions), and we introduce a level-synchronized parallel sampling mechanism that shares progress across trajectories rather than training each trajectory independently.

Test-Time Scaling and Parallel Search. Parallel sampling at test time has been widely studied in code generation through the pass@k framework [Chen et al., 2021], where generating k independent samples increases the probability of producing a correct solution. Recent work has extended test-time compute scaling to reasoning tasks through methods such as Monte Carlo Tree Search (MCTS) [Silver et al., 2017], best-of-n sampling, and iterative self-refinement. However, these methods are primarily designed for single-turn generation and do not directly apply to multi-step interactive tasks where partial progress (e.g., completing intermediate levels) can be shared across trajectories. Our level-synchronized parallel sampling extends pass@k to the multi-step setting, providing a principled mechanism for checkpointing and sharing progress that is complementary to single-turn search methods.

4. Technical Preliminaries

In this section, we first formalize the problem setup for ARC-AGI-3 as a multi-step decision-making task, then introduce the pass@k metric and extend it from single-step code generation to multi-step interactive tasks.

4.1. Problem Setup

We consider a setting in which an LLM agent interacts with an ARC-AGI-3 environment to complete multi-step tasks. Each *game* (e.g., ft09, lp85, vc33) consists of a sequence of *levels* with

increasing difficulty, indexed by $\ell = 1, 2, \ldots, L$. At each discrete time step $t = 1, 2, \ldots$, the agent observes a state $\boldsymbol{s}_t \in \mathcal{S}$ representing the current game screen and selects an action $a_t \in \mathcal{A}$ from a finite discrete action space. The environment then transitions to the next state $\boldsymbol{s}_{t+1}$ and provides a sparse reward $r_t \in \{0, 1\}$ indicating whether the agent has completed the current level.

A *trajectory* $\boldsymbol{\tau} = \{(\boldsymbol{s}_1, a_1, r_1), (\boldsymbol{s}_2, a_2, r_2), \ldots, (\boldsymbol{s}_T, a_T, r_T)\}$ records the full sequence of T interactions within an episode. The agent's behavior is governed by a policy $\pi_\theta(a_t|\boldsymbol{s}_t)$, parameterized by θ, which defines a distribution over actions conditioned on the current state. The objective is to complete all L levels within a step budget of N steps.

Unlike settings with dense per-step rewards, ARC-AGI-3 provides only sparse, verifiable feedback at level completion. As discussed in Section 1, this challenge grows as levels become longer, making level completion increasingly unlikely.

4.2. Single-Step Pass@k

The pass@k metric was originally introduced for evaluating code generation models [Kulal et al., 2019, Chen et al., 2021]. In this *single-step* setting, the model generates k candidate outputs for a given input, and a problem is considered solved if *any* of the k samples passes verification. The unbiased estimator is:

$$\text{pass@}k := \mathbb{E}_{\text{Problems}} \left[1 - \frac{\binom{n-c}{k}}{\binom{n}{k}} \right] \tag{1}$$

where n is the total number of samples generated, and c is the number of correct samples.

4.3. Multi-Step Pass@k

We extend pass@k to *multi-step* agentic tasks where level completion requires a sequence of actions rather than a single generation.

For multi-step tasks, we define pass@k as follows: given a maximum step budget N per episode, we run k independent episodes on the same task. A level is considered completed if at least one of the k episodes completes it. Formally:

$$\text{pass@}k := \mathbb{E}_{\text{Levels}} \left[\mathbf{1} \left[\exists\, i \in [k] : S_i \leq N \right] \right] \tag{2}$$

where S_i denotes the number of steps for episode i to complete the task, with $S_i = \infty$ if it fails, and $[k] = \{1, \ldots, k\}$.

The key difference from single-step pass@k is that each "sample" is now an entire trajectory of actions, not a single output. We refer to this approach of running k parallel trajectories as *parallel sampling*, and describe our level-synchronized implementation in Section 5.

5. Method

Our approach has two main components: a parallel sampling mechanism built on multi-step pass@k (Section 4), and multi-turn RLVR training that improves action selection over successive rounds of interaction.

5.1. Parallel Sampling

To use multi-step pass@k, we design a practical parallel sampling mechanism for ARC-AGI-3.

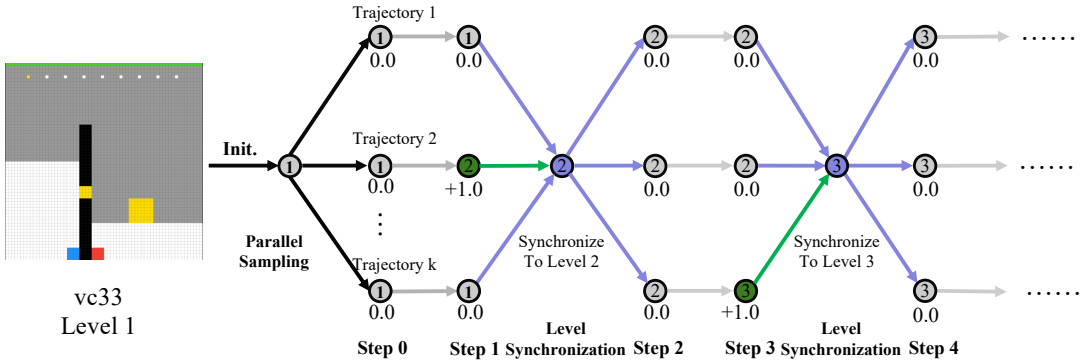


Figure 3 | **Level-synchronized parallel sampling with test-time training on VC33.** We initialize k parallel agents identically at Level 1. Each training step consists of C game actions; a green node with +1.0 reward indicates level completion, while gray nodes with 0.0 reward indicate no completion. The number inside each node denotes the agent's current level. Thanks to multi-step pass@k, running k parallel agents greatly increases the chance that at least one completes the current level. Once one agent completes the level, Level Synchronization shares that progress with all agents, as shown by the purple hub nodes and purple fan-out arrows, so that every agent resumes exploration from the newly reached level rather than starting over.

From Sparse Rewards to Parallel Sampling. As established in Section 1, the sparse reward structure of ARC-AGI-3 makes any single trajectory unlikely to complete a level, since success demands a long sequence of correct actions with no intermediate positive feedback. Running multiple independent trajectories increases the probability that at least one succeeds. The key insight is that we can apply pass@k at the granularity of each level rather than the entire game, synchronizing progress across trajectories to maximize efficiency.

Level-Synchronized Parallel Sampling. As illustrated in Figure 3, we instantiate k parallel agent instances, each operating in its own environment initialized at the first level. Each agent independently attempts to complete the current level through its own sequence of actions. When any agent completes the current level, Level Synchronization occurs: all other agents discard their partial progress and advance to the level-completing state. From this synchronized state, all k agents resume independent exploration on the next level. This process repeats until all levels are completed or exploration terminates.

This synchronization mechanism directly addresses the sparse reward problem by improving sampling efficiency. Running k trajectories in parallel increases the probability that at least one discovers a level-completing action sequence, yielding a positive trajectory that would be rare under single-trajectory sampling. Once any agent completes the current level, that progress is shared across all agents rather than being lost, so the k agents collectively need to find only one solution per level instead of each agent independently solving all levels from scratch. This makes positive reward signals available more frequently and accelerates overall game completion.

Context across levels. When the environment advances to a new level, each agent instance begins from the new level's initial observation. The agent does not carry forward an explicit conversation history or memory buffer from earlier levels. However, knowledge acquired from previous levels is retained in two ways: (1) the policy weights π_θ are updated via RLVR after each round, encoding strategies learned from earlier interactions, and (2) the observation itself often contains cues from earlier levels (e.g., displayed level counters, persistent game elements),

allowing the model to implicitly condition on prior progress through the observation alone.

5.2. Online Multi-Turn RLVR with Trajectory Chunking

Training Overview. We apply multi-turn Reinforcement Learning with Verifiable Rewards (RLVR) to improve action selection in ARC-AGI-3. Our setting is *online reinforcement learning*: given a specific game to solve, the model adapts its policy π_θ by learning from its own interactions with that game. Unlike offline RL pipelines that train on a fixed dataset before deployment, our online approach interleaves policy updates with environment interaction. The model generates rollouts, receives rewards, and refines its action selection, while working to solve the target game.

Following the setup in Section 4, the agent collects trajectories by interacting with the environment, and each trajectory is used to update the policy via RLVR, encouraging actions that lead to level completion. This cycle of rollout and policy update repeats throughout the game.

Trajectory Chunking. Many levels in ARC-AGI-3 require dozens of attempts to complete, with individual episodes spanning 100 or more steps. Running until level completion in a single training step poses two challenges. First, the agent may execute a long sequence of actions without receiving any positive feedback. Second, even when the level is eventually completed, attributing a single reward across an excessively long action sequence makes credit assignment difficult.

To address these issues, we introduce *trajectory chunking*: rather than running until level completion before assigning a reward, we divide each trajectory into segments of at most C actions and evaluate whether the level was completed after each segment. Concretely, the agent executes up to C actions starting from the current state $\boldsymbol{s}_t$ and receives a reward at the end of each segment. As illustrated in Figure 3, each node represents one such segment: a green node with reward +1.0 indicates that the level was completed within that segment, while a gray node with reward 0.0 indicates that it was not. When the level is not completed, the next segment resumes from the resulting state $\boldsymbol{s}_{t+C}$. This bounds the length of each training trajectory, ensuring that when a positive reward is obtained, it can be meaningfully attributed to the actions within that segment. The specific values of k and C are detailed in Section 6.1.

Our full approach combines both mechanisms in an iterative loop. At each round, k parallel agents interact with the environment in chunks of C actions, with level synchronization sharing progress whenever any agent completes a level. The resulting trajectory segments, each annotated with a reward indicating whether level completion occurred, are used to update the policy via RLVR. The improved policy is then deployed for the next round of parallel sampling, improving the agent's action selection over successive rounds.

6. Experiments

6.1. Experiment Setup

Games. We evaluate our approach on four of the six publicly available ARC-AGI-3 games: ft09, lp85, vc33, and as66, as shown in Figure 1. These four games were selected to cover all three control schemes (click-based, arrow-based, and combined) and span a range of difficulty levels. The remaining two games, ls20 and sp80, are excluded from the current evaluation: ls20 uses a small arrow-only action space ($|\mathcal{A}| = 4$) where even the base model makes partial progress, making it less informative for studying sparse reward challenges; sp80 uses a combined control

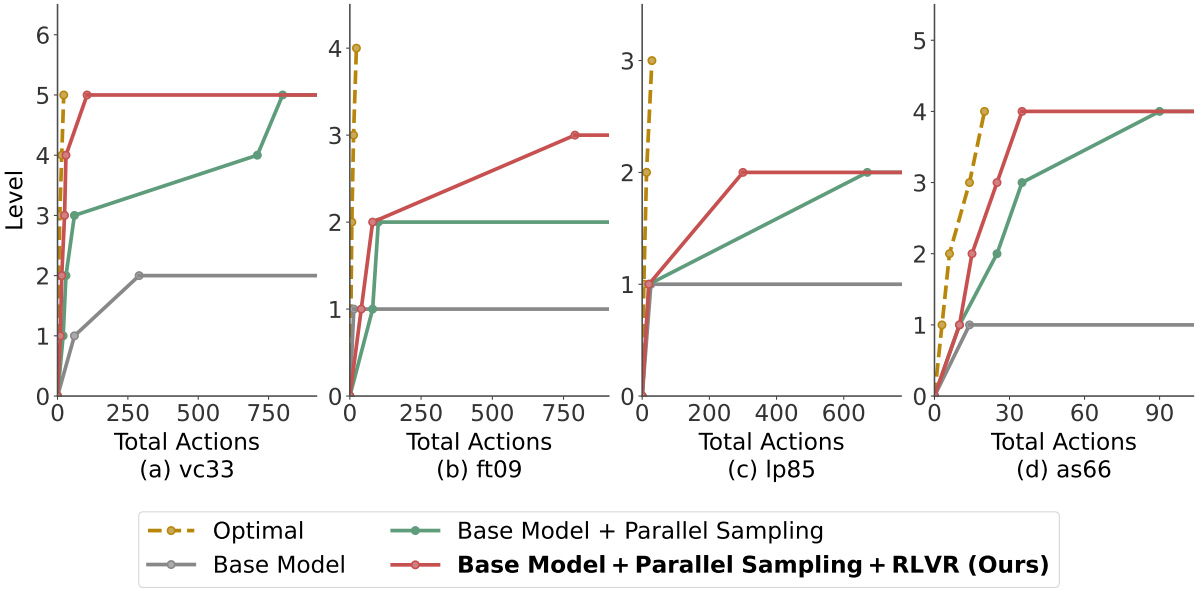


Figure 4 | **Cumulative actions required to reach each level.** For each game, the x-axis shows the total number of actions from game start and the y-axis shows the level reached. Steeper curves indicate faster level completion, meaning the method reaches the same level with fewer actions. Four settings are compared: the base model alone, the base model with parallel sampling, our full method combining parallel sampling with multi-turn RLVR, and the optimal solution requiring the minimum number of actions.

scheme similar to as66, so we prioritized as66 to avoid redundancy. Each game presents distinct challenges in interactive reasoning, requiring the agent to discover game mechanics and execute multi-step action sequences to complete levels.

Training details. We use Qwen2.5-7B-Instruct [Yang et al., 2024] as our base model and fine-tune it with LoRA (rank 64, alpha 64) applied to all linear layers, using a learning rate of 3×10^{-6}. We run $k = 128$ parallel trajectories with level synchronization. For policy optimization, we adopt GRPO [Shao et al., 2024]. In each training round, the k parallel agents produce trajectory segments (each of at most C actions), yielding a group of N segments $\{\boldsymbol{\tau}_1, \ldots, \boldsymbol{\tau}_N\}$. Each segment $\boldsymbol{\tau}_i$ receives a binary reward $R(\boldsymbol{\tau}_i) \in \{0, 1\}$ indicating whether the level was completed within that segment. Following GRPO, we compute the episode-level advantage for each segment by normalizing its reward with respect to the group statistics:

$$\hat{A}(\boldsymbol{\tau}_i) = \frac{R(\boldsymbol{\tau}_i) - \text{mean}\big(\{R(\boldsymbol{\tau}_j)\}_{j=1}^{N}\big)}{\text{std}\big(\{R(\boldsymbol{\tau}_j)\}_{j=1}^{N}\big)}. \tag{3}$$

In standard GRPO for single-turn tasks, the group consists of multiple responses sampled for the same input. In our multi-turn setting, each trajectory segment begins from a potentially different game state due to the parallel sampling and chunking procedure. We therefore treat all trajectory segments from a single training round as one group for advantage computation, since they share the same policy $\pi_{\theta_{\text{old}}}$ and target the same game. The policy is then updated via the clipped surrogate objective with KL regularization:

$$\mathcal{J}(\theta) = \mathbb{E}\left[\frac{1}{N}\sum_{i=1}^{N} \min\Big(\rho_\theta(\boldsymbol{\tau}_i)\,\hat{A}(\boldsymbol{\tau}_i),\ \text{clip}\big(\rho_\theta(\boldsymbol{\tau}_i), 1 \pm \epsilon\big)\,\hat{A}(\boldsymbol{\tau}_i)\Big)\right] - \beta\,\mathbb{D}_{\text{KL}}\big(\pi_\theta \,\|\, \pi_{\text{ref}}\big), \tag{4}$$

where $\rho_\theta(\boldsymbol{\tau}_i) = \pi_\theta(\boldsymbol{\tau}_i)/\pi_{\theta_{\text{old}}}(\boldsymbol{\tau}_i)$ is the importance sampling ratio. We set the KL coefficient $\beta = 0.01$ and the discount factor $\gamma = 0.95$. The trajectory chunk size C is set to 5, limiting each

training segment to at most 5 actions. Training runs for 150 epochs on 8×A100 GPUs. The prompt template used for agent-environment interaction is provided in Appendix B.

Metrics. We evaluate model performance using the cumulative number of actions required to reach each level. This is the official ARC-AGI-3 evaluation criterion [Kamradt, 2025a]: agents are scored primarily on levels completed, with action count as the tiebreaker among equally performing agents. Action efficiency serves as a direct proxy for skill acquisition speed—an agent that learns the game's rules faster wastes fewer actions on incorrect attempts. We report cumulative actions (rather than per-level actions) because it captures both effectiveness (whether a level is reached) and efficiency (how quickly). We acknowledge that compute cost (GPU hours, tokens generated) is an important complementary metric and discuss it further in Section 7.

6.2. Results

Parallel sampling and multi-turn RLVR both contribute to level completion. Figure 4 presents the cumulative actions required to reach each level across the four evaluated games, with detailed results in Table 2. The base model alone completes early levels but stalls quickly: on vc33 it reaches Level 2 in 28 actions but fails to progress further, and on ft09 and lp85 it cannot advance past Level 1. Adding parallel sampling substantially increases the levels reached. On vc33, parallel sampling reduces the actions to reach Level 2 from 28 to 2 and enables completion of all five levels, reaching Level 5 in 79 actions. On ft09, it enables Level 2 completion in 9 actions, whereas the base model cannot do so even after over 1,000. Incorporating RLVR on top of parallel sampling further improves performance. On vc33, the actions required to reach Level 5 decrease from 79 to 19, an approximately 4× reduction. On ft09, RLVR enables Level 3 completion at 78 actions, a level that parallel sampling alone cannot reach. On lp85, RLVR reduces the actions to reach Level 2 from 66 to 29. These results show that the two components provide complementary benefits: parallel sampling extends the set of reachable levels, while RLVR improves action selection within those levels.

Our method generalizes across diverse game mechanics. The four games exhibit different difficulty characteristics and control schemes. vc33 uses click-based control with an action space of 4,096 possible click locations. lp85 also employs click-based control but presents a more challenging exploration landscape, where even parallel sampling struggles beyond Level 2. ft09 combines spatial reasoning with sequential dependencies, making later levels particularly difficult. as66 uses a combined control scheme integrating both arrow and click inputs. Despite these variations, our method consistently improves performance across all games. Parallel sampling provides substantial gains on games where the base model struggles, and RLVR further refines action selection to reduce the total actions required.

Learned capabilities transfer beyond ARC-AGI-3. A natural question is whether the trained models pick up general reasoning skills or simply memorize game-specific patterns. We test this by evaluating transfer to ALFWorld [Shridhar et al.], a text-based household task benchmark with fundamentally different mechanics and observation modalities. As shown in Table 1, all four trained models substantially outperform the baseline on the overall ALFWorld success rate, with improvements ranging from 7.9 to 13.7 percentage points. The transfer patterns differ by game: ft09 excels at Clean tasks (34.4%), vc33 shows strong performance on Heat (21.4%) and Cool (11.1%) tasks, and as66 achieves the highest Look accuracy (55.8%). These complementary strengths point to each game teaching distinct reasoning skills that transfer selectively. The consistent gains across all four models indicate that the learned policies capture general interactive reasoning strategies rather than any single game's mechanics.

Table 1 | **Transfer to ALFWorld.** Models trained on individual ARC-AGI-3 games are evaluated on ALFWorld [Shridhar et al.] to test whether learned capabilities generalize beyond the training game.

Model	**ALFWorld**						
	Pick	Look	Clean	Heat	Cool	Pick2	All
Baseline	33.4	21.6	19.3	6.9	2.8	3.2	14.8
Ours (ft09)	60.6	43.6	**34.4**	6.3	9.4	0.0	**28.5**
Ours (lp85)	44.3	47.8	27.2	3.1	5.6	3.1	22.7
Ours (vc33)	**60.7**	41.7	20.0	**21.4**	**11.1**	4.5	26.6
Ours (as66)	50.9	**55.8**	26.4	19.6	9.3	**5.4**	27.7

7. Conclusion

We have presented an approach for solving ARC-AGI-3, an interactive reasoning benchmark that poses fundamental challenges for current AI systems due to novel game mechanics and sparse learning signals. The two core techniques—multi-turn RLVR, which learns from level-completing trajectories online, and level-synchronized parallel sampling, which synchronizes progress across multiple trajectories—work together to reach levels the base model cannot and cut action counts by up to 4×. Transfer results on ALFWorld further show that the learned skills are not confined to the training games, pointing toward agents that can efficiently adapt to novel tasks through exploration and sequential decision-making.

Limitations and future work. Our approach has several limitations that suggest directions for future work. First, level-synchronized parallel sampling relies on deterministic environments: when any agent completes a level, all others synchronize to that state, which assumes replaying the same actions from a given state yields the same outcome. Extending this to stochastic environments would require maintaining a distribution over reachable states or developing approximate synchronization strategies. Second, running $k = 128$ parallel trajectories on 8×A100 GPUs incurs substantial compute cost. A thorough analysis of how performance scales with the number of parallel agents and the associated computational budget would help practitioners balance cost against performance. Third, we evaluate on a single base model (Qwen2.5-7B-Instruct) and four of the six ARC-AGI-3 games. Evaluating across model families and scales, as well as the remaining games, would strengthen the generality claims. Finally, the trajectory chunk size $C = 5$ and parallel agent count $k = 128$ were chosen based on preliminary experiments; systematic ablation studies over these hyperparameters would provide clearer guidance for applying the method to new tasks.

Acknowledgments

We would like to extend special thanks to Greg Kamradt and the ARC Team for their help. We also gratefully acknowledge Lambda for providing a research grant that supported the computational resources used in this work.

References

ARC Prize Foundation. ARC-AGI-3: Interactive reasoning benchmark. https://arcprize.org/arc-agi/3/, 2026. Accessed: 2026.

Kevin Chen, Marco Cusumano-Towner, Brody Huval, Aleksei Petrenko, Jackson Hamburger, Vladlen Koltun, and Philipp Krähenbühl. Reinforcement learning for long-horizon interactive llm agents. *arXiv preprint arXiv:2502.01600*, 2025.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

François Chollet. On the measure of intelligence. *arXiv preprint arXiv:1911.01547*, 2019.

François Chollet and Mike Knoop. Francois chollet + mike knoop: Fireside chat. ARC Prize @ MIT, 2025. Video available at https://www.youtube.com/watch?v=RsmSkmGjHso.

Francois Chollet, Mike Knoop, Gregory Kamradt, Bryan Landers, and Henry Pinkard. ARC-AGI-2: A new challenge for frontier AI reasoning systems, 2026. URL https://arxiv.org/abs/2505.11831.

Bowen Jin, Hansi Zeng, Zhenrui Yue, Jinsung Yoon, Sercan Arik, Dong Wang, Hamed Zamani, and Jiawei Han. Search-r1: Training llms to reason and leverage search engines with reinforcement learning. *arXiv preprint arXiv:2503.09516*, 2025.

Greg Kamradt. ARC-AGI-3 and action efficiency. ARC Prize @ MIT, 2025a. Video available at https://www.youtube.com/watch?v=bqNfIHedb3g.

Greg Kamradt. Interactive reasoning benchmarks: ARC-AGI-3 preview. ARC Prize @ MIT, 2025b. Video available at https://www.youtube.com/watch?v=3T4OwBp6d90.

Sumith Kulal, Panupong Pasupat, Kartik Chandra, Mina Lee, Oded Padon, Alex Aiken, and Percy S Liang. Spoc: Search-based pseudocode to code. In H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc., 2019. URL https://proceedings.neurips.cc/paper/2019/file/7298332f04ac004a0ca44cc69ecf6f6b-Paper.pdf.

Karthik Narasimhan, Tejas Kulkarni, and Regina Barzilay. Language understanding for text-based games using deep reinforcement learning. In *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing*, pages 1–11, 2015.

OpenAI. Gpt-5 system card. *openai.com/index/gpt-5-system-card*, 2025.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Mingchuan Zhang, YK Li, Yu Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Cote, Yonatan Bisk, Adam Trischler, and Matthew Hausknecht. Alfworld: Aligning text and embodied environments for interactive learning. In *International Conference on Learning Representations*.

David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas Baker, Matthew Lai, Adrian Bolton, et al. Mastering the game of Go without human knowledge. *Nature*, 550(7676):354–359, 2017.

Hado Van Hasselt, Arthur Guez, and David Silver. Deep reinforcement learning with double q-learning. In *Proceedings of the AAAI conference on artificial intelligence*, volume 30, 2016.

Zihan Wang, Kangrui Wang, Qineng Wang, Pingyue Zhang, Linjie Li, Zhengyuan Yang, Xing Jin, Kefan Yu, Minh Nhat Nguyen, Licheng Liu, et al. Ragen: Understanding self-evolution in llm agents via multi-turn reinforcement learning. *arXiv preprint arXiv:2504.20073*, 2025.

Zhiheng Xi, Jixuan Huang, Chenyang Liao, Baodai Huang, Honglin Guo, Jiaqi Liu, Rui Zheng, Junjie Ye, Jiazheng Zhang, Wenxiang Chen, et al. Agentgym-rl: Training llm agents for long-horizon decision making through multi-turn reinforcement learning. *arXiv preprint arXiv:2509.08755*, 2025.

An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, et al. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.

A. Additional Experimental Results

Model	Level 1	Level 2	Level 3	Level 4	Level 5
vc33					
Optimal	2	4	8	15	22
Base Model	5	28	>1000	>1000	>1000
Base Model + Parallel Sampling	1	2	5	70	79
Base Model + Parallel Sampling + Multi-Turn RLVR **(Ours)**	**0**	**1**	**4**	**15**	**19**
ft09					
Optimal	4	7	13	23	33
Base Model	13	>1000	>1000	>1000	>1000
Base Model + Parallel Sampling	7	9	>100	>100	>100
Base Model + Parallel Sampling + Multi-Turn RLVR **(Ours)**	**3**	**7**	**78**	>100	>100
lp85					
Optimal	5	13	29	>29	>29
Base Model	25	>1000	>1000	>1000	>1000
Base Model + Parallel Sampling	**1**	66	>100	>100	>100
Base Model + Parallel Sampling + Multi-Turn RLVR **(Ours)**	**1**	**29**	>100	>100	>100
as66					
Optimal	3	6	14	20	>150
Base Model	13	>1000	>1000	>1000	>1000
Base Model + Parallel Sampling	**1**	4	6	17	>150
Base Model + Parallel Sampling + Multi-Turn RLVR **(Ours)**	**1**	**2**	**4**	**6**	>150

Table 2 | **Cumulative actions required to reach each level on ARC-AGI-3 games.** We compare four settings: the optimal solution (minimum possible actions), the base model alone, the base model with parallel sampling, and the full method combining parallel sampling with multi-turn RLVR (Ours). Lower values indicate faster level completion. **Bold** numbers indicate the best model performance in each column within each game.

B. Prompt Template

Prompt Template for ARC-AGI-3

 # CONTEXT
You are an agent playing a dynamic game. Your objective is to WIN and avoid GAME_OVER while minimizing actions.

One action produces one Frame. One Frame is made of one or more sequential Grids. Each Grid is a matrix of size 64×64, each cell represents a visual component of the game.

 # HISTORY
Prior to this step, you have already taken {step_count} step(s). Below are the most recent {history_length} actions you took:
{action_history}

 # CURRENT STEP
You are now at step {current_step}.

 # CURRENT GRID
<image>

 # GAME STATE
Current score: {score}

 # INTERACTIVE COMPONENTS
{component_context}

 # TURN
Based on your action history, the current grid, and interactive components, choose one interactive component to interact with to advance toward your game objective.

 # ACTION FORMAT
You must output ONLY the coordinates where you want to click, in the format (x, y) where:
- x and y are integers between 0 and 63 (inclusive)
- The grid is 0-indexed, so (0, 0) is top-left and (63, 63) is bottom-right
- Coordinates outside this range are invalid and will cause errors
Do not include any other text, just the coordinates.

Example response:
<think>
[Your detailed analysis of your previous actions, the current board state, review of interactive components and their positions, review of game state, and choose one interactive component to interact with to advance toward your game objective.
</think>
<answer>(4, 32)</answer>

Figure 5 | Prompt template used for the LLM agent interacting with ARC-AGI-3 environments. Template variables are shown in brown. The agent outputs its reasoning within <think> tags and its action within <answer> tags.